

OpenCV 5 - Python Cheat Sheet

Fast reference for images, video, processing, contours, features, geometry and DNN inference

OpenCV 5.0.0 • Python 3

0. Start / inspect

```
import cv2 as cv
import numpy as np

print(cv.__version__)
print(img.shape)      # (height, width, channels)
print(img.dtype)       # usually uint8
h, w = img.shape[:2]

| OpenCV color images are BGR by default, not RGB.
```

1. Read / write / color

```
img = cv.imread("in.jpg", cv.IMREAD_COLOR)
gray = cv.imread("in.jpg", cv.IMREAD_GRAYSCALE)
cv.imwrite("out.png", img)

gray = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
rgb = cv.cvtColor(img, cv.COLOR_BGR2RGB)
hsv = cv.cvtColor(img, cv.COLOR_BGR2HSV)

| Check img is None after imread() before processing.
```

2. Resize / crop / transform

```
small = cv.resize(img, (640, 480),
                  interpolation=cv.INTER_AREA)
up = cv.resize(img, None, fx=2, fy=2,
              interpolation=cv.INTER_CUBIC)

crop = img[y1:y2, x1:x2]
flip = cv.flip(img, 1)          # 1 horizontal
rot = cv.rotate(img, cv.ROTATE_90_CLOCKWISE)

M = cv.getRotationMatrix2D((w/2, h/2), 15, 1.0)
rot = cv.warpAffine(img, M, (w, h))

# mask / ROI
roi = cv.bitwise_and(img, img, mask=mask)
```

3. DNN / ONNX inference

```
net = cv.dnn.readNetFromONNX("model.onnx")
blob = cv.dnn.blobFromImage(
    img, scaleFactor=1/255.0,
    size=(640,640), swapRB=True, crop=False)
net.setInput(blob)
out = net.forward()

| OpenCV 5: ONNX is the preferred interchange path; Caffe/
Darknet loaders were removed.
```

4. Blur / threshold / morphology

```
g = cv.GaussianBlur(gray, (5, 5), 0)
m = cv.medianBlur(gray, 5)

_, binary = cv.threshold(gray, 127, 255,
                        cv.THRESH_BINARY)
otsu_t, otsu = cv.threshold(gray, 0, 255,
                          cv.THRESH_BINARY + cv.THRESH_OTSU)

adapt = cv.adaptiveThreshold(gray, 255,
                             cv.ADAPTIVE_THRESH_GAUSSIAN_C,
                             cv.THRESH_BINARY, 11, 2)

k = cv.getStructuringElement(cv.MORPH_RECT, (3,3))
opened = cv.morphologyEx(binary, cv.MORPH_OPEN, k)
closed = cv.morphologyEx(binary, cv.MORPH_CLOSE, k)
eroded = cv.erode(binary, k, iterations=1)
dilated = cv.dilate(binary, k, iterations=1)
```

5. Edges & contours

```
edges = cv.Canny(gray, 100, 200)

contours, hierarchy = cv.findContours(
    binary, cv.RETR_EXTERNAL,
    cv.CHAIN_APPROX_SIMPLE)

for c in contours:
    area = cv.contourArea(c)
    peri = cv.arcLength(c, True)
    x, y, w, h = cv.boundingRect(c)
    approx = cv.approxPolyDP(c, 0.02 * peri, True)

cv.drawContours(img, contours, -1, (0,255,0), 2)

| Use a binary/edge image for findContours().
```

6. Draw / annotate

```
cv.line(img, (x1,y1), (x2,y2), (255,0,0), 2)
cv.rectangle(img, (x,y), (x+w,y+h), (0,255,0), 2)
cv.circle(img, (cx,cy), r, (0,0,255), -1)
cv.putText(img, "label", (x,y-8),
           cv.FONT_HERSHEY_SIMPLEX, 0.7,
           (255,255,255), 2, cv.LINE_AA)
```

7. Camera / video loop

```
cap = cv.VideoCapture(0) # or "video.mp4"
if not cap.isOpened():
    raise RuntimeError("Cannot open video")

while True:
    ok, frame = cap.read()
    if not ok: break

    cv.imshow("frame", frame)
    if cv.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv.destroyAllWindows()

| OpenCV 5: unsupported VideoCapture.get() properties
return a negative value; test with < 0.
```

8. Features & matching

```
# ORB: binary descriptors -> Hamming
orb = cv.ORB_create(nfeatures=1000)
kp, des = orb.detectAndCompute(gray, None)

bf = cv.BFMatcher(cv.NORM_HAMMING,
                 crossCheck=True)
matches = bf.match(des1, des2)
matches = sorted(matches, key=lambda m: m.distance)

# SIFT: float descriptors -> L2
sift = cv.SIFT_create()
kp, des = sift.detectAndCompute(gray, None)
```

9. Geometry & pose

```
H, inliers = cv.findHomography(src_pts, dst_pts,
                              cv.RANSAC, 5.0)
warped = cv.warpPerspective(img, H, (out_w, out_h))

ok, rvec, tvec = cv.solvePnP(
    object_pts, image_pts, K, dist_coeffs)

proj, _ = cv.projectPoints(points3d, rvec, tvec,
                          K, dist_coeffs)
```

OpenCV 5 - What Changed

Migration map for OpenCV 4.x → 5.0.0, plus practical compatibility notes

Released June 6, 2026

Build & language baseline

C++17+ OpenCV 5 requires C++17 or later. The migration guide lists GCC 8 (7.x with caveats), Clang 9, and MSVC 2017 (19.14) or later.

Python 3 Python 2 support is removed; use Python 3.6+.

Legacy C API removed CvMat, IplImage, cvCreateMat(), cvFindContours() and the old OpenCV 1.x C API are gone.

```
# CMake
set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
```

Module restructuring (C++)

Functionality	4.x	5.x
Calibration	calib3d	calib
Stereo	calib3d	stereo
Homography / PnP	calib3d	geometry
Point cloud / TSDF	partly rgbd	ptcloud
Features	features2d	features

```
#include <opencv2/geometry.hpp>
#include <opencv2/calib.hpp>
#include <opencv2/stereo.hpp>
#include <opencv2/features.hpp>
```

<opencv2/calib3d.hpp> remains as an umbrella compatibility header. Python still uses cv2.<func>().

Moved to opencv_contrib

ml **gapi** moved out of the main repository.

Haar / HOG CascadeClassifier and HOGDescriptor moved to xobjdetect.

Some features SURF, BRIEF, FREAK, LUCID, DAISY and others are in xfeatures2d. SIFT, ORB, FAST, GFTT and MSER remain mainline.

```
cmake -DOPENCV_EXTRA_MODULES_PATH=/path/to/opencv_contrib/
modules /path/to/opencv
```

Core data model changes

True 1D arrays: std::vector<T> now maps to a true 1D Mat, not an N×1 2D column matrix. Prefer mat.total() for element count.

MatShape: preferred over MatSize for new code; compatibility remains.

New types: CV_16BF, CV_32U, CV_64U, CV_64S, CV_Bool.

DNN engine & model loading

New default engine dnn::readNet() tries the new engine first and can fall back to the classic engine.

Removed readNetFromDarknet() and readNetFromCaffe(). Convert models to ONNX.

```
net = cv.dnn.readNetFromONNX("model.onnx")
```

The new engine is CPU-only in 5.0. For GPU inference, use the classic engine or a GPU-enabled ONNX Runtime build.

Image-processing behavior changes

Nearest resize: INTER_NEAREST and INTER_NEAREST_EXACT now follow the same convention and align with Pillow; old pixel baselines can change.

Warping: warpAffine, warpPerspective and remap use revised interpolation. APIs are unchanged, but pixels can differ slightly from 4.x.

Text rendering

TrueType / UTF-8 New FontFace-based rendering supports UTF-8 and custom .ttf/.otf fonts.

Legacy FONT_HERSHEY_* calls remain, but appearance/metrics may differ from 4.x.

Video I/O compatibility

Unsupported VideoCapture.get() properties now return a negative value instead of the ambiguous 0.

```
val = cap.get(cv.CAP_PROP_AUTOFOCUS)
if val < 0:
    # unsupported
    ...
```

Property support depends on backend → OS → driver → hardware.

Python migration in one glance

- Most cv2 call sites remain unchanged despite C++ module restructuring.
- ONNX DNN code generally stays the same; Caffe/Darknet loaders are gone.
- NumPy/InputArray code may need adjustment if it assumed an N×1 matrix.
- cv2.ml.*, Haar and HOG may require a contrib build.

Upgrade debug checklist

- | | |
|----------------------------|-----------------------------------|
| Build fails? | Confirm C++17/compiler level. |
| Header missing? | Check module rename/contrib move. |
| Shape assertion? | Inspect 1D-vs-2D assumptions. |
| DNN loader gone? | Convert to ONNX. |
| Pixel tests fail? | Review resize/warp baselines. |
| Camera prop < 0? | Treat as unsupported. |

Version sanity checks

```
# Python
print(cv.__version__)
```

```
// C++
#include <opencv2/core/version.hpp>
std::cout << CV_VERSION << "\n";
```

When behavior surprises you, confirm the runtime version and backend before debugging your algorithm.

OpenCV 5 - New APIs & Algorithms

OpenCV 5.0-only additions and changed behavior - practical reference

5.0 ONLY • NEW / CHANGED

Modern local features + ANN

ALIKED **DISK** **LightGlue** Deep local features and a learned matcher are now part of the modern features direction.

Annoy ANN Approximate-nearest-neighbor search based on Annoy replaces the old standalone FLANN path.

```
#include <opencv2/features.hpp>
```

TRUCO contours - automatic acceleration

OpenCV 5 adds the threaded TRUCO contour extractor. `findContours()` uses it automatically when possible - the call stays the same.

```
contours, hierarchy = cv.findContours(  
    binary, cv.RETR_EXTERNAL,  
    cv.CHAIN_APPROX_SIMPLE)
```

Main benefit: faster contour extraction without a new API.

0D / 1D Mat + new element types

```
std::vector<float> v{1,2,3};  
cv::Mat m(v); // 5.x: true 1D Mat  
// dims=1, rows=1, cols=3  
// use m.total() for vector length
```

CV_16BF **CV_32U** **CV_64U** **CV_64S** **CV_Bool**

CV_Bool uses one byte/value and is accepted as a mask where 8-bit masks are valid. **CV_16BF** supports conversion to/from **CV_32F**.

LAPACK is always available for SVD, eigen work and USAC; OpenCV builds a small internal subset when external LAPACK is absent.

OpenCV 5 cleanup

- **OpenVX removed.**
- Classic **ML** and **G-API** moved to `opencv_contrib`.
- Haar/HOG moved to `xobjdetect`.
- Many obsolete samples were removed/revised.

DNN engine selection

ENGINE_AUTO OpenCV tries the new engine first and can fall back to classic.

```
net = cv.dnn.readNet("model.onnx")
```

```
net = cv.dnn.readNetFromONNX(  
    "model.onnx", engine=cv.dnn.ENGINE_CLASSIC)  
# ENGINE_ORT in ORT-enabled builds  
Env override: OPENCV_FORCE_DNN_ENGINE: 1=classic, 2=new, 3=auto,  
4=ORT.
```

ONNX Runtime integration / GPU route

OpenCV 5 can link ONNX Runtime while using OpenCV's own ONNX parser.

```
cmake -DWITH_ONNXRUNTIME=ON \  
      -DDOWNLOAD_ONNXRUNTIME=ON ...
```

```
# NVIDIA execution provider  
cmake -DWITH_ONNXRUNTIME=ON \  
      -DDOWNLOAD_ONNXRUNTIME_GPU=ON ...
```

The new native DNN engine in 5.0 is CPU-only. For GPU, use classic DNN or an ORT GPU build.

VLMs + ONNX-first model workflow

The new DNN stack includes tokenizers, attention, decoding blocks, post-processing and KV-cache components for end-to-end vision-language model inference.

Caffe/Darknet parsers removed Convert migrated models to ONNX. TFLite remains available through the classic engine.

```
cd <opencv>/samples/dnn  
python3 download_models.py
```

TrueType / UTF-8 text engine

```
font = cv.FontFace("sans")  
# built-ins: "sans", "italic", "uni"  
Custom .ttf/.otf fonts and broad Unicode support are new. C++ also gains alignment flags. Complex shaping (for example Arabic/Devanagari) and color emoji remain limited.
```

USAC: default robust-geometry backbone

USAC is the default robust-estimation framework for homography, essential/fundamental matrices, PnP and related geometry.

Toolbox: PROSAC/NAPSAC sampling, MSAC/MAGSAC++ scoring, LO-RANSAC/GC-RANSAC local optimization and SPRT early rejection. Parallel execution is supported.

Expect stronger behavior on noisy, outlier-heavy data than classic RANSAC baselines.

Multi-view camera calibration

New N-camera pipeline: per-camera intrinsics → pairwise extrinsics → global optimization.

Pinhole **Fisheye** **Mixed rigs**

Checkerboard, ChArUco and circle-grid targets are supported, including partial observations when cameras have limited overlap.

Dedicated point-cloud / mesh direction

ptcloud High-level 3D processing now has a dedicated module.

Volume	TSDF integration
Registration	ICP
I/O	.ply / .obj
Motion	visual odometry foundations

Migration smoke test

```
print(cv.__version__)  
assert cv.__version__.split('.')[0] == '5'
```

```
v = cap.get(cv.CAP_PROP_AUTOFOCUS)  
unsupported = (v < 0)  
Re-baseline pixel-exact tests for nearest-neighbor resize, warps and text: OpenCV 5 intentionally changes those outputs/metrics.
```